PYTHON COURSE — DAY 47 REFERENCE

Environment Variables & Flask Configuration | 25 May 2026

Session 1 — Environment Variables and .env Files

Hardcoded secrets in code are dangerous — pushed to GitHub, anyone can read them. Environment variables store secrets outside the code.

```
# .env file — never commit to GitHub SECRET_KEY=kalikeng_secret_key_2026_secure_version
DATABASE_URL=kalikeng.db JWT_EXPIRY_HOURS=24 DEBUG=True
```

```
from dotenv import load_dotenv import os load_dotenv() # reads .env file into os.environ secret =
os.environ.get('SECRET_KEY') # None if not set database = os.environ.get('DATABASE_URL',
'kalikeng.db') # default value hours = int(os.environ.get('JWT_EXPIRY_HOURS', '24')) # convert to
int debug = os.environ.get('DEBUG', 'False').lower() == 'true' # convert to bool print(secret) #
kalikeng_secret_key_2026_secure_version print(database) # kalikeng.db print(hours) # 24
print(debug) # True
```

```
# .gitignore — files Git ignores .env __pycache__/ *.pyc books.db kalikeng.db
```

Business Use: When the Kalikeng tool is deployed to a cloud server — the server has its own environment variables with production secrets. The same code runs in development (your laptop) and production (cloud) — only the .env values differ.

Key Rules:

• pip install python-dotenv

• load_dotenv() — call before accessing any environment variables

• os.environ.get('KEY', 'default') — always provide a default value

• .env must be in .gitignore — never push secrets to GitHub

• int(os.environ.get('HOURS', '24')) — env vars are strings, convert as needed

• os.environ.get('DEBUG', 'False').lower() == 'true' — string to boolean

• git add .gitignore — not .env

Session 2 — Flask Configuration

```
from dotenv import load_dotenv import os load_dotenv() app = Flask(__name__)
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'fallback') app.config['DATABASE'] =
os.environ.get('DATABASE_URL', 'kalikeng.db') app.config['JWT_HOURS'] =
int(os.environ.get('JWT_EXPIRY_HOURS', '24')) app.config['DEBUG'] = os.environ.get('DEBUG',
'False').lower() == 'true' # Use in functions token = jwt.encode(payload,
app.config['SECRET_KEY'], algorithm='HS256') payload = jwt.decode(token,
app.config['SECRET_KEY'], algorithms=['HS256'])
```

Config Classes — professional pattern:

```
class Config: SECRET_KEY = os.environ.get('SECRET_KEY', 'fallback') DATABASE =
os.environ.get('DATABASE_URL', 'kalikeng.db') JWT_HOURS = int(os.environ.get('JWT_EXPIRY_HOURS',
'24')) DEBUG = False class DevelopmentConfig(Config): DEBUG = True # debug on in development
class ProductionConfig(Config): DEBUG = False # debug off in production config = { 'development':
DevelopmentConfig, 'production': ProductionConfig, 'default': DevelopmentConfig } env =
os.environ.get('FLASK_ENV', 'development') app.config.from_object(config[env])
```

Business Use: The taxi tool runs in development mode on your laptop (DEBUG=True, shows error details). The same code deployed to the cloud uses ProductionConfig (DEBUG=False, hides error details from users). One codebase, two environments.

Key Rules:

• app.config['KEY'] — access configuration anywhere in the app

• Config class — all settings in one place

• DevelopmentConfig — DEBUG=True, for local development

• ProductionConfig — DEBUG=False, for live server

• FLASK_ENV env variable — switches between configs automatically

• SECRET_KEY must be at least 32 characters for JWT — use long strings

• Never expose SECRET_KEY in API responses

Session 3 — Applying Config to Auth System

Session 3 updates the auth system to use proper configuration — no more hardcoded values anywhere.

```
# Database helper — write once, use everywhere def get_db(): conn =
sqlite3.connect(app.config['DATABASE']) # from config not hardcoded conn.row_factory =
sqlite3.Row return conn # Usage — always call with brackets conn = get_db() # correct — calls the
function conn = get_db # WRONG — this is the function object, not the result
```

```
# Updated create_token — uses app.config def create_token(username, role): payload = { 'user':
username, 'role': role, 'exp': datetime.datetime.now(datetime.timezone.utc) +
datetime.timedelta(hours=app.config['JWT_HOURS']) } token = jwt.encode(payload,
app.config['SECRET_KEY'], algorithm='HS256') return token # Updated verify_token — uses
app.config def verify_token(token): try: payload = jwt.decode(token, app.config['SECRET_KEY'],
algorithms=['HS256']) return payload except Exception as e: print(f'JWT Error: {e}') return None
```

Business Use: When the SECRET_KEY changes (security best practice — rotate keys periodically), you only update the .env file. No code changes needed. When deploying to a different server with a different database path — only .env changes.

Key Rules:

• get_db() — helper function, call with brackets always

• All sqlite3.connect() calls replaced with get_db()

• create_token uses app.config['SECRET_KEY'] and app.config['JWT_HOURS']

• verify_token uses app.config['SECRET_KEY']

• load_dotenv() must be called before app.config lines

• One .env file change = updates everywhere in the app

• KeyError on app.config means the key was never set — check config setup